

SOAP y REST: cómo hacer que el software hable entre sí

Sesión de puras diapositivas, sin código en vivo. El profesor abre formalmente la Unidad II (servicios y APIs) con una pregunta: si un sistema mezcla Java, .NET, PHP, Android y iPhone, ¿cómo los pones a platicar? Responde con dos estilos de servicio web, SOAP y REST, y termina asomándose al primer controlador de Spring, que es justo lo que se construye en la clase siguiente.

La página lleva los dos protocolos en color: **SOAP** en violeta, el contrato formal, y **REST** en naranja, el ligero. Los botones con **?** explican los términos que dio por sabidos. Hay seis ejercicios intercalados.

Lo que salió de esta clase

≈00:00

Opcional: la **aplicación de propósito general** (Práctica 1). Dijo que no es obligatoria, pero que quien quiera adelantarla puede. Vuelve a salir al ≈19:20.

≈15:05

Arrastre de la clase 5: los **códigos de respuesta HTTP**. Los volvió a nombrar como algo que se sigue respetando; sigue sin fecha.

≈35:05

Pendiente administrativo: nos vemos el **próximo miércoles en el laboratorio**.

aclaración

En esta sesión **no dictó ninguna tarea nueva con fecha**. El checklist del final junta las tres cosas de arriba más un recordatorio de XML.

i

Sobre los minutos y la fecha. La transcripción de esta clase no traía tiempos reales: las marcas ≈ están estimadas por el ritmo del habla (cada 1 o 2 minutos), así que valen ±1 minuto. La fecha tampoco viene. Por lo que dice el repo, esta clase es la 7 y va antes de la del 9 de septiembre. Confírmala con tu calendario antes de usarla para otra cosa.

BLOQUE 1 · MINUTO ≈00:00 A 03:22

Por qué existe SOAP

Abre la Unidad II retomando la aplicación de propósito general que ya mencionó antes, y de ahí salta a la pregunta de fondo: cómo se comunican piezas hechas con tecnologías distintas.

"Formalmente dando carpetazo a la de inicio a la unidad dos. Que tiene que ver con servicios, APIs y demás. Entonces, como primera escala, vamos a platicar tantito de SOAP. Que es, literal, el protocolo simple de acceso a objetos."

La idea de partida: tengo un problema que resolver con varios tipos de software y varios lenguajes, así que necesito una forma de **ponerlos a platicar**. Si lo logro, ya no dependo de una base de datos, de un sistema operativo ni de un lenguaje en particular. Para lograrlo se apoya en **protocolos estándar**, es decir, reglas que todos aceptan.

– ¿Qué significa "sistemas heterogéneos"?

Sistemas hechos con tecnologías distintas: uno en Java, otro en .NET, otro en Python, con bases de datos y sistemas operativos diferentes. Lo contrario es un sistema **homogéneo**, donde todo usa la misma tecnología. Lo difícil de la integración es que cada pieza "habla" un formato distinto.

Aclaró que SOAP se enfoca sobre todo en el **ámbito empresarial**, porque cumple con los principios **ACID** del manejo de transacciones. Al recitarlos se trabó con el orden y él mismo se corrigió con un "bueno, en el orden correcto".

– ¿Cuáles son los principios ACID, en el orden correcto?

- **Atomicidad:** la transacción se hace completa o no se hace.
- **Consistencia:** la base pasa de un estado válido a otro válido.
- **Aislamiento:** dos transacciones simultáneas no se estorban entre sí.
- **Durabilidad:** lo confirmado no se pierde, aunque se caiga el sistema.

Él dijo "atomicidad, consistencia, durabilidad, aislamiento" y se corrigió; el orden de las siglas es el de arriba. Lo recordamos de cuando jugamos con bases de datos.

- i **Precisión mía, no del profesor.** Él dice que SOAP "le da cumplimiento" a ACID. Para el examen conviene decirlo así: SOAP se asocia con transacciones confiables. Ese soporte lo aportan las extensiones del estándar (la familia WS-*, como WS-AtomicTransaction), no el mensaje SOAP básico por sí solo.

BLOQUE 2 · MINUTO ≈ 03:22 A 05:25

Web Services: el intermediario entre lenguajes

Definió el servicio web y lo aterrizó con un sistema de facturación imaginario.

"Es aquel conjunto de normas, de reglas, de protocolos, de estándares, que nos van a poder permitir hacer que una aplicación escrita en .NET, por ejemplo, se comuniquen con una aplicación en Java, con una aplicación en Python o con cualquier cosa que se les ocurra."

El ejemplo fue el típico **sistema de facturación** con módulos en PHP, Java, .NET, Python y Go, que además corre en varias plataformas: Android, iPhone y HarmonyOS de Huawei, más web, escritorio y consola. Hay dos caminos para que se hablen: escribir tú un **middleware**, o usar servicios. Un servicio web hace el trabajo de un middleware.

— ¿Qué es un middleware?

Software que se coloca **en medio** de dos aplicaciones para traducir o coordinar lo que se dicen. Cada aplicación habla con el middleware en vez de hablar directo con la otra, y el middleware sabe cómo tratar con cada una.

SOAP · EL CONTRATO FORMAL

BLOQUE 3 · MINUTO ≈ 05:25 A 08:08

El WSDL: el contrato

Para usar un servicio web hace falta un convenio entre las dos partes. Ese convenio es un documento XML.

*"Necesitamos un convenio, de alguna manera, entre las dos partes. Y ese convenio no es otra cosa más que un documento basado en XML.
[...] Se llama **Web Service Descriptor Language, WSDL**, que es el contrato."*

La analogía fue la renta de un cuarto: quien ha rentado firmó un convenio con reglas (no llegar muy noche, no hacer escándalo, cuidar los espacios). Esas reglas son obligatorias para ambos. En un servicio web, el WSDL cumple ese papel: dice qué se debe seguir.

Concretamente, el WSDL dice:

- cómo se llama el **método**;
- qué **tipo de dato** espera recibir;
- qué **devuelve**;
- cómo se lo vamos a llamar.

Sirve para que las aplicaciones se pongan de acuerdo en el **formato de recepción y de respuesta**. Todo funciona por **paso de mensajes**, y lo que viaja es "una cadenota XML".

*"¿Han hecho interfaces en Java? Y no interfaces gráficas de usuario, sino el **public interface**. El contrato, el que les dice el 'qué' se tiene que hacer. Bueno, pues es lo mismo."*

– ¿Por qué compara el WSDL con un `public interface` ?

Una interfaz de Java declara qué métodos existen, con qué parámetros y qué devuelven, pero no dice **cómo** se hacen. Quien la implementa decide el cómo. El WSDL hace lo mismo entre programas: fija el "qué" y deja libre el "cómo" a cada lado, sin importar en qué lenguaje esté hecho.

Al fondo de este bloque se le complicó cambiar de diapositiva con el proyector ("no quiere"). Fue una pausa breve, sin contenido.

BLOQUE 4 · MINUTO ≈08:08 A 10:43

Los cinco estándares que sostienen a SOAP

SOAP funciona porque se apoya en estándares que todos aceptan. Los fue nombrando uno por uno.

ESTÁNDAR	PARA QUÉ SIRVE, SEGÚN ÉL
HTTP	El protocolo de transferencia de hipertexto: "todos hemos agarrado el HTTP" para enviar datos y hacer peticiones.
SOAP	El protocolo del que trata la clase: define cómo se arma el mensaje.
WSDL	El descriptor: "dice qué podemos hacer con respecto al servicio web".
XML	El formato en que se envían los datos que esperamos recibir.
XSD	Define los elementos que se esperan dentro del XML: qué tipo de dato es, si es obligatorio, si es lista, si tiene asociaciones. Son las reglas de validación del documento.

*"Ustedes recordarán en sus clases que les dijeron que un XML tendría que ser **bien formado y validado**. La buena formación, pues, un montón de reglas que debe de cumplir. La validez, lo que le da el peso al documento, es cómo está conformado, la manera en que aparecen cada uno de los elementos."*

– ¿Qué diferencia hay entre "bien formado" y "válido"?

Bien formado: cumple las reglas de sintaxis de XML. Todas las etiquetas cierran, hay un solo elemento raíz, los atributos van entre comillas. Cualquier lector de XML lo acepta.

Válido: además de estar bien formado, respeta las reglas de **un esquema concreto** (el XSD): que exista tal elemento, en tal orden, con tal tipo de dato. Un documento puede estar bien formado y no ser válido.

Al presentar el WSDL (bloque 3) el profesor pidió recordar la tercera y cuarta unidad de **Tecnologías para el Desarrollo de Aplicaciones Web**, donde se vio XML.

Lo que se logra con todo esto: sin importar en qué esté desarrollado tu proyecto, los distintos lenguajes van a poder intercambiar información. Bromeó con no saber si el proyector fallaba por el adaptador o por el cable; sin contenido técnico.

EJERCICIO 1 • QUIÉN HACE QUÉ

Los estándares de SOAP

Cuatro preguntas sobre el papel de cada pieza. Contesta las cuatro y dale a **Revisar**.

¿Qué documento es el contrato que fija método, tipos de entrada y de salida?

WSDL

XSD

HTTP

¿Cuál define las reglas de validación de los elementos del XML (tipo, si es obligatorio, si es lista)?

WSDL

XSD

SOAP

¿En qué formato viajan los datos de un mensaje SOAP?

JSON

XML

PDF

Un documento XML tiene todas sus etiquetas cerradas y un solo elemento raíz, pero no respeta el XSD. ¿Cómo está?

Bien formado, pero no válido

Válido, pero mal formado

Ni bien formado ni válido

Revisar

Reiniciar

REST • EL LIGERO

BLOQUE 5 • MINUTO ≈ 10:43 A 12:30

REST, JSON y por qué el XML sigue vivo

Entra el otro estilo. También es un conjunto de principios para intercambiar información, pero con otra forma de mensaje.

Dice la gente que el XML ya no se usa y que deberíamos usar JSON. El profesor explica la razón: el documento sigue siendo "una cadena de texto", pero JSON nace de JavaScript y es fácil de leer tanto para máquinas como para humanos. Ganó auge porque se necesitaba transferir datos sobre todo para **dispositivos móviles y servicios web ligeros**.

Luego preguntó si seguimos usando XML. La respuesta fue que sí, y el ejemplo fue la factura:

*"¿Ustedes han facturado algo alguna vez? ¿Cuál es la factura? El XML. El XML es la factura. Literal. [...] Ustedes a lo mejor dicen 'ah, no, es que mi factura es el PDF'. No es cierto, esa es la **representación impresa** de tu factura. Tu factura es el XML."*

— ¿Quién es "la entidad recaudadora más importante de México"?

No la nombró. Es el SAT. El comprobante fiscal digital (CFDI) es un archivo XML; el PDF que te llega es una representación impresa. Por eso, dijo, "tenemos que darle cumplimiento" al XML en los sistemas de facturación.

BLOQUE 6 · MINUTO ≈ 12:30 A 15:23

REST se apoya en HTTP

REST no inventa un protocolo nuevo. Aprovecha lo que ya trae HTTP.

Usa los **verbos** de HTTP para cada acción. Los que normalmente usamos:

GET, POST, PUT, DELETE y PATCH. Sirven para recuperar uno o muchos, insertar, hacer actualizaciones totales o parciales, y borrar. Puede devolver los resultados en HTML, XML o lo que se te ocurra, pero **lo típico es JSON**.

*"En el sentido estricto, cuando jugamos con REST, lo único que tenemos que hacer es seguir todo el conjunto de reglas que pudiera existir dentro del manejo del mismo protocolo. [...] Sabemos que el HTTP por definición es un **protocolo sin estado**, entonces a ustedes les toca encontrar la forma de garantizar la **trazabilidad** de las peticiones."*

— ¿Qué quiere decir que HTTP no tiene estado?

Cada petición llega sola: el servidor no recuerda por sí mismo quién eres ni qué pediste antes. Si necesitas que una serie de peticiones se reconozcan como parte de lo mismo (una sesión, un carrito, un usuario ya autenticado), tienes que construir esa continuidad tú: con sesiones, cookies, tokens, etc. A eso se refiere con "trazabilidad".

También se siguen respetando los **códigos de respuesta y de estado**, "sobre todo para indicar si la petición fue satisfactoria, si es un recurso prohibido,

si hay variedad de opciones y demás".

ARRASTRE · CLASE 5 ≈15:05

Códigos de respuesta del protocolo HTTP

Aquí no dictó nada nuevo, pero volvió a nombrarlos. En la clase 5 los pidió con un "se requieren, desafortunadamente" y sin fecha; sigue sin haberla.

"Se siguen respetando los códigos de respuesta y los códigos de estado."

BLOQUE 7 · MINUTO ≈15:23 A 19:20

API, gateway y sistemas legados

Definió qué es una API, dibujó cómo se organiza y cerró con la pregunta de cuándo usar SOAP y cuándo REST.

*"Por definición es una **interfaz de programación de aplicaciones**, y la regla es simple: es aquel conjunto de reglas, de normas, de protocolos que vamos a usar para poder integrar software de aplicaciones."*

En la diapositiva aparecen dos piezas. El **Backend System** es lo que construimos del lado del servidor. El **API Management System**, o **API Gateway**, es el responsable de "determinar qué se puede hacer, quién puede responder y redireccionar al espacio correspondiente". Delante pueden estar aplicaciones de escritorio, web, móviles, de consola o dispositivos IoT, y todas se integran mientras exista manera de comunicarse. Ejemplo: sensores que disparan una acción y guardan datos; al usuario final solo le importa ver "numeritos" o una gráfica.

— ¿Qué hace un API Gateway?

Es la puerta de entrada única a tus servicios. Recibe todas las peticiones, decide si el solicitante puede hacerlas, y las manda al servicio interno correcto. Así los clientes no necesitan conocer la dirección de cada servicio.

La diferencia entre los dos

'Sistemas legados. Esas cosas raras que existen, que funcionan, sepa la... cómo. Donde ni siquiera hay documentación, o más bien la documentación pues ya se fue. ¿Y qué es más fácil? ¿Mantenerlos o actualizarlos? Típicamente trabajan con SOAP.'

	SOAP	REST
Formato del mensaje	XML, "sí o sí"	JSON. También puede recibir y generar XML
Dónde lo verás	Sistemas legados; empresas muy formales; donde los esquemas de seguridad son un requisito de cumplimiento	Servicios web ligeros, Internet de las cosas, aplicaciones móviles, <i>serverless</i>
Carácter	Verboso, con contrato (WSDL)	Ligero, apoyado en los verbos de HTTP

Su conclusión práctica: cuando hagas cosas "muy empresariales, muy formales", lo más seguro es que tengas que lidiar con SOAP.

– ¿Qué es *serverless* ?

Un modelo de nube donde subes funciones o pequeños servicios y el proveedor se encarga de los servidores: los levanta cuando llega una petición y cobra por uso. No significa que no haya servidores, sino que tú no los administras. El profesor solo lo listó como uno de los lugares donde REST es ideal.

¿SOAP o REST?

Cuatro escenarios con los criterios que dio. La última pregunta cierra un malentendido común.

Una empresa formal exige cumplir esquemas de seguridad estrictos en sus servicios. ¿Qué es más probable que use?

☐ SOAP☐ REST

Una aplicación móvil necesita traer datos rápido, con mensajes pequeños. ¿Qué encaja mejor?

☐ SOAP☐ REST

Hay un sistema viejo que funciona, sin documentación, y hay que conectarlo con otra cosa. ¿Con qué suele venir?

☐ SOAP☐ REST

¿Es cierto que un servicio REST solo puede trabajar con JSON?

☐ Sí, solo JSON☐ No: también puede recibir y generar XML

BLOQUE 8 · MINUTO ≈19:20 A 24:14

La aplicación de propósito general, por dentro

Retomó el resultado esperado de la práctica y contó cuántas piezas habría que hacer sin servicios, y cómo cambia con ellos.

*"¿Se acuerdan de la aplicación de propósito general? El resultado que esperamos obtener es esta cosa, a final de cuentas, pero eso es **interfaz gráfica de usuario**. ¿Cómo lo van a hacer? No tengo ni la menor idea."*

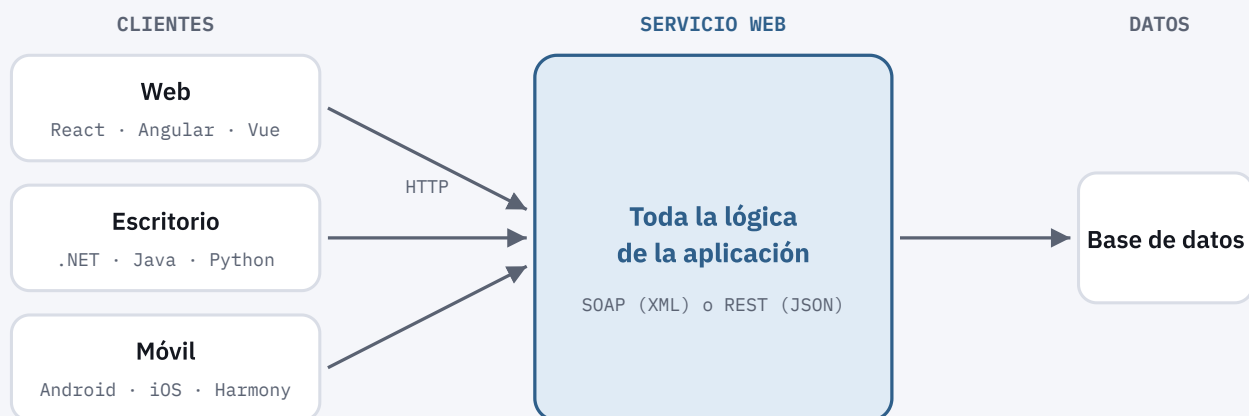
Fue enumerando opciones. Para web: React, Angular o Vue, "un montón de componentes visuales". Para escritorio "nos casamos con la tecnología": Python, .NET o Java, cada uno con su librería gráfica. En .NET se genera un ejecutable o un instalador (un MSI). En Java, con Swing o JavaFX se genera un `.jar`, "el equivalente" de un ejecutable. Sobre Python bromeó: "me han contado que se puede", porque él ya no lo usa.

Y para llevarlo a web hay que instalar el servidor que pida cada tecnología:

- **ASP:** al menos un IIS (Internet Information Services), que ya viene en Windows y solo se habilita.
- **Java con JSP o un framework de Java:** un Tomcat, como mínimo.
- **PHP:** Apache HTTP Server, Nginx o incluso IIS. Los servidores se pueden intercambiar entre tecnologías, pero hay que instalar y configurar.

Sumó además la app para Android, la de iOS y la de Harmony: "tendría como 9 aplicaciones de algún modo". Ahí entra la idea central del bloque:

*"El cliente, párenle de contar, los necesita sí o sí. [...] Pero lo que sí podemos quitar es **toda la lógica de la aplicación**: la podemos meter en un servicio basado en web. Y hacer que esas cosas se puedan poner a platicar entre ellas."*



Cada cliente conserva solo lo visible (botones, menús, cuadros). Lo que antes se repetía en nueve aplicaciones vive una sola vez en el servicio.

Su ejemplo: una aplicación Android escrita en Kotlin, "muy nativa", consume el servicio. Si el servicio es SOAP hace falta un **kSOAP** para trabajar con él. Si es REST hace falta un cliente REST, como el **RestTemplate** de Spring. Al final mencionó un "HTTP Template", pero no quedó claro si era otra cosa o una corrección del mismo nombre; se lo deja como dijo.

– ¿Qué son kSOAP y RestTemplate?

kSOAP es una librería ligera para que una aplicación Android pueda armar y leer mensajes SOAP. **RestTemplate** es la clase de Spring que sirve para llamar a servicios REST desde Java: le das la URL y el verbo y te devuelve la respuesta ya convertida a objeto. Ambos son el "cliente" del servicio: el lado que consume.

La aplicación de propósito general (Práctica 1)

No la dictó como tarea. Al principio de la clase dijo, casi de pasada, que no la tienen que hacer, y que quien quiera adelantarla, adelante. El resultado esperado es una interfaz gráfica; la tecnología es libre.

"Práctica 1 o algo así. Eso, supuestamente. No lo tienen que hacer, por cierto, pero bueno. Sí, sí, claro, si lo quiere hacer, adelante."

SOAP · EL MENSAJE POR DENTRO

BLOQUE 9 · MINUTO ≈24:14 A 26:45

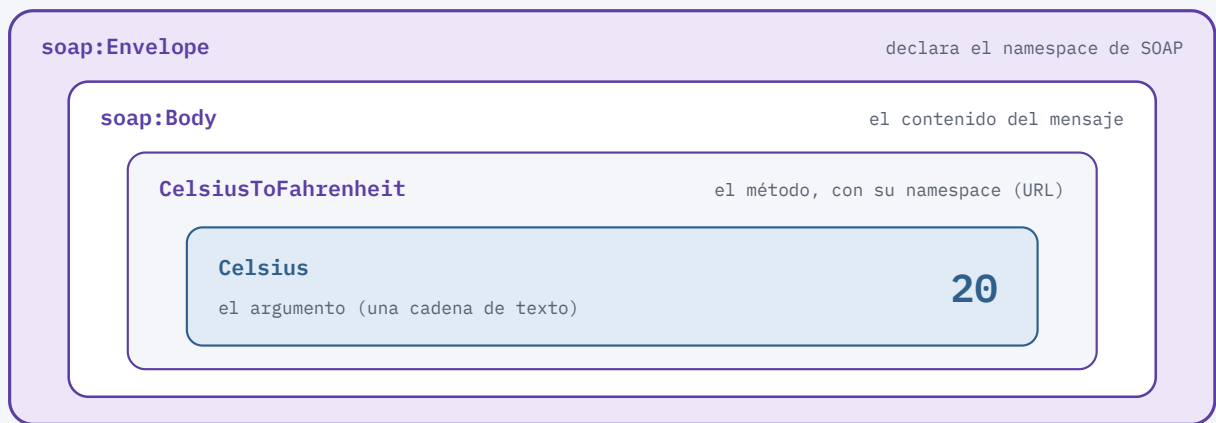
Una petición SOAP, etiqueta por etiqueta

Mostró en diapositiva el ejemplo de conversión de temperatura de W3Schools y lo fue leyendo de arriba abajo.

La lectura fue esta: primero el **XML** con su versión y codificación. Luego el **SOAP Envelope**, donde se declara el **namespace**, es decir, la norma con las reglas de validación de los esquemas. Después el **SOAP Body**, con el método `CelsiusToFahrenheit` y su propio namespace apuntando a una URL. Dentro va el argumento `Celsius` con el valor `20`. Y se cierra todo en orden inverso: argumento, función, cuerpo, envelope.

Así queda la petición. Ojo: el profesor la **describió leyéndola**, no la dictó línea por línea. Yo la reconstruyo con esa descripción y el ejemplo de W3Schools que citó, así que los namespaces exactos son los de ese ejemplo:

```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Body>
    <CelsiusToFahrenheit xmlns="https://www.w3schools.com/xml/">
      <Celsius>20</Celsius>
    </CelsiusToFahrenheit>
  </soap:Body>
</soap:Envelope>
```



Cuatro niveles, cada uno dentro del anterior. Por eso se cierra en orden inverso, y por eso dicen que SOAP es "verboso": para pedir un número hace falta este envoltorio.

*"¿Por qué dicen que es **verboso**? Porque en realidad esta es la petición que ustedes envían cuando arman o quieren consumir un servicio en SOAP."*

Sobre el argumento, aclaró que `Celsius` se declara como **string**: "porque a final de cuentas es una cadena de texto. Está basado en XML".

La **respuesta** sigue el mismo patrón: el método se llama `CelsiusToFahrenheitResponse`, el valor viaja en `CelsiusToFahrenheitResult` y también es una cadena de texto. Si abres el enlace de la diapositiva puedes ver el **WSDL completo** de ese servicio.

```
<CelsiusToFahrenheitResponse xmlns="https://www.w3schools.com/xml/">
  <CelsiusToFahrenheitResult>68</CelsiusToFahrenheitResult>
</CelsiusToFahrenheitResponse>
```

- i Conexión con la clase siguiente.** El resultado `68` es el mismo que el profesor usa como valor de prueba el 9 de septiembre (`20 °C → 68 °F`) cuando arma el mismo conversor como API REST. Es el mismo problema resuelto con las dos tecnologías, y vale la pena compararlos.

La petición SOAP, en orden

Toca las líneas en el orden en que aparecen, de arriba abajo. Si te equivocas te doy una pista; toca una línea ya puesta para quitarla.

Aquí se va armando el mensaje...

<Celsius>20</Celsius>

<?xml version="1.0" encoding="utf-8"?>

<CelsiusToFahrenheit xmlns="https://www.w3schools.com/xml/">

<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">

</CelsiusToFahrenheit>

<soap:Body>

</soap:Envelope>

</soap:Body>

Reiniciar

BLOQUE 10 · MINUTO ≈ 26:45 A 29:43

Los verbos de REST y el nivel de madurez

Del lado de REST, lo primero es asociar cada verbo con las cuatro operaciones que de siempre hacemos: insertar, borrar, actualizar y seleccionar.

VERBO	OPERACIÓN	CUÁNDO LO USÓ ÉL
GET	Seleccionar	Para devolver uno o para devolver todos.
POST	Insertar	Crear un recurso nuevo.
PUT	Actualizar	Solo si la actualización es completa .
PATCH	Actualizar	Cuando la actualización es parcial : cambiar un solo atributo.
DELETE	Eliminar	Borrar el recurso.

La razón de PATCH la explicó con un usuario recién creado:

*"Se creó el usuario originalmente. Y hasta que no le pique a la liga que le llegó al correo electrónico, o al código que mandan, el **OTP**, hasta ese momento no se activa el usuario. Digamos que se crea pero en un estatus de falso. Para cambiar una variable en particular, un atributo, utilizamos **PATCH**."*

Y si usas los cinco verbos, con versionado y documentación, ya le "estás pegando" a otro concepto: el **nivel de madurez de Richardson** para APIs. "Para empezar deberíamos cumplir los cinco, más otra cierta serie de características."

– ¿Qué es el modelo de madurez de Richardson?

Una escala que mide qué tan "REST de verdad" es una API, en cuatro niveles del 0 al 3. El profesor solo nombró el modelo; el resumen es mío:

1. **Nivel 0:** un solo punto de entrada y HTTP usado solo como transporte.
2. **Nivel 1:** aparecen los **recursos**, cada uno con su propia URL.
3. **Nivel 2:** se usan bien los **verbos** HTTP y los códigos de estado.
4. **Nivel 3:** la respuesta trae enlaces que dicen qué se puede hacer después (HATEOAS).

La estructura del mensaje es la misma que en SOAP, "solo que en lugar de apuntar al XML, apuntamos a un JSON. Y san se acabó". Y consumirlo es más transparente: para interactuar basta con **apuntar al protocolo HTTP**, no hace falta más. En Spring existe `RestTemplate` ; en Java puro, o en cualquier lenguaje, basta con llamar a los métodos de HTTP.

EJERCICIO 4 · ELIGE EL VERBO

De la operación al verbo HTTP

Cinco situaciones. Elige el verbo que él asociaría a cada una.

Devolver un usuario, o devolverlos todos.

GET

POST

PUT

PATCH

DELETE

Dar de alta un usuario nuevo.

GET

POST

PUT

PATCH

DELETE

Reemplazar todos los datos de un registro existente.

GET

POST

PUT

PATCH

DELETE

Cambiar solo el estatus de un usuario, de falso a verdadero, cuando confirma su código OTP.

GET

POST

PUT

PATCH

DELETE

Quitar un registro.

GET

POST

PUT

PATCH

DELETE

Revisar

Reiniciar

BLOQUE 11 · MINUTO ≈29:43 A 31:26

Documentar una API: Swagger y RAML

Antes de construir, hay que poder decirle a otros qué hace tu API.

*"La forma más rápida de documentar un API es agarrar **Swagger**. [...] Y si lo quieren hacer más detallado, más elaborado, o batallar tantito... pues siempre tenemos el **RAML** (RESTful API Modeling Language)."*

Lo que busca es cumplir con **estándares aceptados** para generar la documentación, y Swagger lo hace. La pantalla que se ve enumera cada método y cómo interactuar con él. Ejemplos de lo que dijo que aparece:

- GET /user para devolver a todos los usuarios;
- GET /user/{id} , con un argumento que sirve de ID para la consulta;
- para insertar, se pasa un objeto;

- para actualizar, se pasa el objeto **más el ID**, "comúnmente".

— ¿Qué son Swagger y RAML, y en qué se parecen?

Las dos son formas de **describir una API REST** en un archivo estándar: qué rutas tiene, con qué verbos, qué parámetros recibe y qué devuelve. A partir de ese archivo se genera una página de documentación navegable (la de Swagger permite hasta probar cada método). Swagger es la más popular y la más rápida; RAML es más elaborada y orientada a modelar la API con detalle.

BLOQUE 12 · MINUTO ≈ 31:26 A 35:05

El primer controlador en Spring

Cierra asomándose a lo que se construye en la siguiente clase: un controlador REST en Java con Spring, y por qué no debe ser tan optimista.

Creas un controlador y lo decoras con `@RestController`, "porque es un API REST". Le pones un nombre para llegar a él con `@RequestMapping`. Luego defines un método por verbo, cada uno con su anotación: `@PostMapping`, `@GetMapping`, `@PutMapping`, `@PatchMapping`, "cada una de esas cosas". La del borrado no la nombró en voz alta, pero es `@DeleteMapping`.

Después no solo importa qué dato devuelves, sino **qué código de respuesta le pintas al usuario**. Para eso está `@ResponseStatus` con un `HttpStatus`:

SI EL MÉTODO...	EL <code>HTTPSTATUS</code> ES
borra	NO_CONTENT
crea	CREATED
actualiza	OK
consulta	OK

"¿Qué otra cosa? Nada más. Con esos tenemos."

El problema: es muy "happy path"

Así como está, el controlador asume que **todo siempre sale bien**: se ejecuta, hace todo, devuelve y listo. No se valida nada. Pero rara vez ocurre así, y los códigos de respuesta los va a controlar el alumno. El ejemplo que dio, con la carrera:

"Voy a dar de alta una carrera para ESCOM, y la carrera que voy a dar de alta es ISC. El mismo nombre ya está registrado. Entonces no me

debería de permitirlo. Por lo tanto, debería de botarme alguna excepción."

Otros casos que nombró: no se logró conectar a la base de datos, ocurrió un error interno del servidor, no se encontró el recurso, o el recurso no está permitido para ese usuario con esos privilegios. "Todo eso los tenemos que manejar nosotros."

– ¿Qué es un "happy path"?

El camino en el que todo sale bien: los datos llegan correctos, la base responde, el usuario tiene permiso. El código de "happy path" solo cubre ese camino. El código de verdad tiene que prever los **errores**: datos duplicados, fallos de conexión, recursos que no existen.

Por eso las firmas simples como `public Evento read()` o `public Evento save(@RequestBody Evento e)` no deberían quedar así. `@RequestBody` hace el mapeo del JSON recibido a la clase, pero lo que se devuelve no debería ser la entidad sola:

"Lo que deberíamos de devolver no es la entidad en sí. Tendríamos que devolver un `ResponseEntity`. Para poder interactuar con él, y a lo mejor poder determinar si se puede o si no se puede hacer tal o cual cosa."

- i **Lo que no se llegó a ver.** No escribió el controlador completo ni el manejo de excepciones; solo lo enseñó en diapositiva y lo nombró. El código de verdad aparece la clase siguiente (9 de septiembre), donde sí usa `ResponseEntity` en vivo.

EJERCICIO 5 · ESCRIBE EL CÓDIGO

@ResponseStatus(HttpStatus.____)

Escribe la constante que él puso para cada método. Vale con o sin el prefijo `HttpStatus.`.

Un método que **borra** un recurso.

@ResponseStatus(HttpStatus.) Comprobar

Un método que **crea** un recurso.

@ResponseStatus(HttpStatus.) Comprobar

Un método que **actualiza** (o consulta) un recurso.

@ResponseStatus(HttpStatus.) Comprobar

Reiniciar

EJERCICIO 6 · EL CIERRE

Happy path y `ResponseEntity`

Tres preguntas sobre por qué el controlador simple se queda corto.

¿Por qué dice que el controlador simple es "muy happy path"?

Porque asume que la operación siempre sale bien y no valida nada

Porque usa demasiadas anotaciones

Porque solo funciona con SOAP

Intentas dar de alta la carrera ISC y ya existe una con ese nombre. ¿Qué debería pasar?

Que se guarde otra igual

Que el sistema lo rechace con una excepción

Que se borre la anterior

¿Qué debería devolver un método del controlador en lugar de solo la entidad?

Un String con el mensaje

Un ResponseEntity

Un void

Revisar

Reiniciar

Cierre de la clase

"Señores, yo hasta aquí le dejo porque ya voy retrasado. Aquí le paramos hoy. ¿Vale? Nos vemos el próximo miércoles, y nos vemos en el laboratorio."

- ! **Pasaje irrecuperable.** La transcripción se corta en mitad de una frase: "Voy a una revisión de...". No sé qué iba a decir después, y no lo invento. Si fue una indicación para el laboratorio, pregúntasela al profesor.

PENDIENTE ADMINISTRATIVO ≈35:05

Laboratorio del próximo miércoles

No dictó entregable. La cita es solo para el laboratorio del miércoles siguiente.

"Nos vemos el próximo miércoles, y nos vemos en el laboratorio."

CIERRE

Checklist de la clase 7

Se guarda en tu navegador. Nada se sube a ningún lado ni se comparte con nadie más.

0 / 4

☐ **Opcional · la aplicación de propósito general (Práctica 1)**

No la dictó como tarea: dijo que no es obligatoria, pero que quien quiera adelantarla puede. Es una aplicación con interfaz gráfica; la tecnología de cliente es libre.

☐ **Pendiente administrativo · laboratorio del próximo miércoles**

Cierre de la clase: "nos vemos el próximo miércoles, y nos vemos en el laboratorio". Lo que iba a decir después ("voy a una revisión de...") se cortó en la transcripción.

☐ **Arrastre · códigos de respuesta del protocolo HTTP**

Viene de la clase 5. Aquí volvió a decir que se siguen respetando los códigos de respuesta y de estado, y que los controlas tú. Sigue sin fecha.

☐ **Recordatorio · repasar XML bien formado y válido**

No es tarea. Pidió recordar la tercera y cuarta unidad de Tecnologías para el Desarrollo de Aplicaciones Web, donde se vio XML, porque WSDL y XSD se apoyan en eso.